

Do learned surrogates, vision models and VLMs catch simulation failures?

A pre-registered benchmark on finite-element trajectories

Muhammad Abdullah

Independent researcher, Melbourne, Australia · abdullahfast95@gmail.com

Technical report, September 2026 · code and results: github.com/ReallyAbdullah/mgn-anomaly

Abstract. Automated checks of simulation output are proposed at every level, from learned surrogates that flag frames their prediction disagrees with, to vision and vision-language models (VLMs) that read renders and explain them. We test whether they work, and against what baseline. We inject synthetic failures modelled on real crash-simulation failure modes into DeepMind’s `deforming_plate` finite-element trajectories: hourglassing, contact penetration, element inversion, numerical instability, frozen regions, and whole-run lag, time-scale and response-scale errors. Each has a known type, location and severity. We compare a noise-trained MeshGraphNet residual detector, a shape-from-load surrogate, one-line physics rules, CLIP-family, SigLIP2 and DINOv2 vision models, and Qwen-family VLM explanations. Every main analysis was pre-registered in version control before it ran, and the test set was scored once. On 92 test simulations, **simple rules match or beat the learned surrogate for every failure type except frozen regions**, and a post-hoc validation check shows that a velocity-Laplacian rule matches it there too. Whole-run loading errors are caught by contact rules or by nothing: a 1–3 frame actuator-plate lag is undetectable by every method tested. No vision model reaches AUROC 0.80, and higher resolution does not help. A local 8B VLM labels 199 of 200 frames “no anomaly”, and counterfactual pairs show its explanations are not grounded in the image. A 27B model from the same family localizes the flagged region but still misnames the failure (2 of 20 correct). We release the benchmark and argue that learned detectors of simulation failures must be reported against their zero-output baseline and cheap physics rules.

1 Introduction

A finite-element (FE) crash simulation can fail silently. Under-integrated elements develop zero-energy *hourglass* modes, contact algorithms let nodes *penetrate* surfaces, elements *invert*, time integration becomes *unstable*, and constraints can fail and *freeze* part of the model. Engineers catch these with energy-ratio checks and by inspecting animations. As simulation volumes grow, and as simulation datasets are assembled to train ML surrogates, there is pressure to automate this QA with learned models. Three families are natural candidates:

- a **learned surrogate** whose prediction residual flags implausible states
- **vision models** on rendered frames
- **vision-language models** that could also explain what went wrong

Whether these methods beat simple baselines is rarely tested. The critique of weak baselines in ML for PDEs [1] applies here directly: a surrogate-residual detector has a trivial *zero-output* counterpart, constant-velocity extrapolation, and there are one-line spatial rules that any engineer could write. We ask:

1. Do learned surrogates detect FE failures better than their zero-output baseline and cheap physics rules?
2. Can generic and modern vision backbones detect the same failures from renders?
3. Do VLM explanations of flagged frames name the right failure, and are they grounded in the image?

Contributions.

- (i) An injected-failure benchmark on public FE trajectories, with ground-truth type, location and severity, and an onset/sustained split.
- (ii) A controlled comparison of surrogate, rule, vision and VLM detectors in one protocol.
- (iii) A pre-registered, gate-by-gate study design: every decision rule was committed to git before its data were seen, and the test set was scored once.
- (iv) A counterfactual grounding test for VLM explanations on engineering renders.

Most of our findings are negative. We report them as such.

2 Related work

ML checks of FE and crash results. Plausibility checking of structural FE results with deep learning [2], [3] learns plausible-vs-implausible from existing simulations, focusing on static modelling errors. Crash-simulation data mining at Fraunhofer SCAI and automotive partners finds outliers and bifurcations across bundles of runs [4]. These address behavioural outliers rather than numerical failure modes. To our knowledge, no prior benchmark has ground-truth numerical failures on FE trajectories or compares learned detectors against physics rules.

Simple baselines. Weak baselines inflate claims in ML for PDEs [1]. In time-series anomaly detection, simple detectors match deep ones [5], and many benchmarks are solvable “with one line of code” [6]. We apply the same standard to surrogate-residual detection.

Vision and VLMs on engineering output. WinCLIP [7] and PatchCore [8] are standard zero- and few-shot visual anomaly detectors, building on CLIP [9] and self-supervised backbones such as DINOv2 [10]. Multimodal LLMs fall short of industrial anomaly-detection requirements [11] and answer simulation questions poorly from images [12]. LLM explanations of crash results have been proposed without checking faithfulness [13]. We test grounding directly with counterfactual pairs.

3 Benchmark

Data. We use DeepMind’s MeshGraphNets `deforming_plate` [14]: a quasi-static, hyperelastic plate (COMSOL, tetrahedral mesh, 750–1,700 nodes per simulation) pressed by a moving actuator, with 400 frames per trajectory. The actuator accelerates about $9\times$ near frame 360, so we model and score frames below 350. Splits are by simulation:

- train 0–249
- validation 20–49 for selection, 50–69 for fitting and reference, 70–99 for calibration
- test 8–99, scored once (0–7 were used in pilots)
- a second unseen set, training trajectories 250–349, reserved for replication

Injected failures. Only plate nodes are changed; the actuator and clamp keep their kinematics. See Table 1. Severities are $1\times$, $10\times$ and $100\times$ the simulation’s RMS per-step node displacement (≈ 0.3 mm). Inversion is geometric: severity 1 distorts the element without flipping it. For frozen regions the severity is the duration. The whole-run errors change the plate response over the entire run.

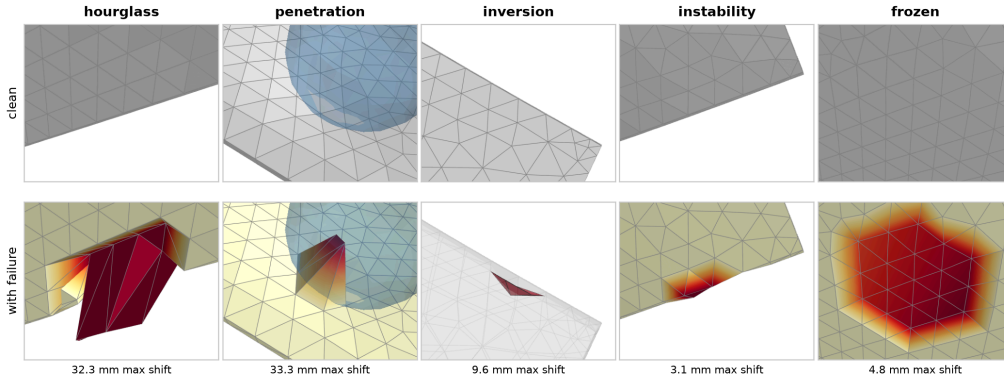


Figure 1: The five local failures at severity 3 (validation simulation 20), close up from the side where each occurs; top clean, bottom with the failure, coloured by deviation. Translucent: actuator; for inversion the plate is see-through and the inverted element solid red. The vision detectors’ fixed camera does not see the actuator side.

Protocol. For every simulation $\times$ type $\times$ severity there is one corrupted copy. All anomalous frames are scored, along with an equal number of clean frames from the same copy, drawn within ± 40 frames of the event but outside a ± 3 -frame guard band. Metrics are frame AUROC and node AUROC. 95% CIs are bootstrapped over simulations, and none are reported with fewer than 20 simulations. GNN-vs-baseline differences use a paired bootstrap with Holm correction.

Failure	Mimics	Injection
hourglass	zero-energy modes	checkerboard offsets on a 2-hop patch, 5 frames
penetration	contact failure	plate nodes in contact pushed through the actuator surface by the severity depth
inversion	negative Jacobian	one tet vertex moved towards its opposite face
instability	numerical blow-up	sign-alternating oscillation growing $1.6\times$ per frame
frozen	constraint/BC error	a region holds its position for 3, 10 or 30 frames
scale / lag / timescale	output or sync errors	whole-run: plate displacement $\times(1+\epsilon)$, k frames late, or at $c\times$ rate

Table 1: Injected failure modes. Frame labels come from the corrupted positions: a frame is anomalous if any node deviates. The first frame of an event is *onset* and the rest are *sustained*.

4 Detectors

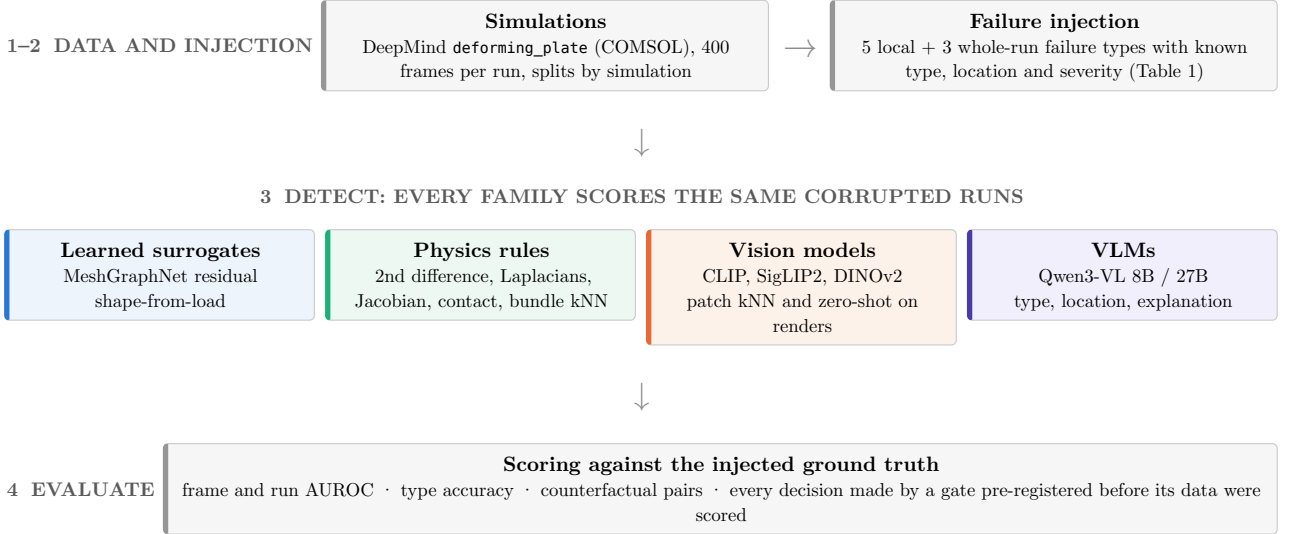


Figure 2: Study pipeline.

Learned surrogate.

- A MeshGraphNet [14] written from scratch in PyTorch on Apple MPS: 10 message-passing steps, latent size 128, 2.6M parameters, with mesh and actuator-contact edge sets.
- It follows the cloth formulation: current velocity in, next acceleration out. Constant-velocity extrapolation is therefore exactly its zero-output baseline.
- Trained 8 h on 250 trajectories with input noise $\sigma = 3\times 10^{-4}$. The noise teaches it to pull perturbed states back towards equilibrium, and was chosen on validation (Gate A).
- A second seed reproduces the clean one-step error: 4.73 vs 4.85×10^{-5} .
- The detection score is the residual $|\text{observed} - \text{predicted}|$, normalised per node.

A second, **shape-from-load** surrogate predicts total plate displacement from the actuator position alone, so a whole-run error can't reach it through its inputs.

Physics rules (no learning):

- constant-velocity second difference
- position Laplacian (displacement minus neighbour mean)
- velocity Laplacian
- velocity z-score
- inverted-element (Jacobian) count
- contact penetration depth and gap

Causal variants use only past frames.

Vision.

- WinCLIP-style zero-shot and patch-kNN [7], [8] on renders with a fixed camera per simulation.
- Backbones: CLIP ViT-B/16 [9], OpenCLIP ViT-L/14, SigLIP2 [15] and DINOv2-L [10].
- Memory banks are built from clean validation renders.

VLMs.

- Qwen3-VL-8B (4-bit, local, MLX) [16], and a pilot with Qwen3.8-27B via OpenRouter.
- Input: the render plus a panel coloured by the surrogate residual, optionally with numeric diagnostics.
- Output: JSON with type, 3×3 grid cell and explanation.

5 Pre-registered gates

Every gate’s question, data, statistic and decision rule was committed before its data were scored (docs/gate_*.md). See Table 2.

Gate	Question	Outcome
A	Training noise $\sigma = 10^{-3}$ vs 3×10^{-4} (validation)	keep 3×10^{-4} ($\Delta -0.105$ [$-0.129, -0.077$])
B	Surrogate one-step error vs constant velocity	$6.9 \times$ worse on clean frames
C	GNN vs best baseline, sustained frames, 9 cells (test)	non-causal: no benefit; causal: $+0.023$, from frozen only
D	Surrogates vs rules on whole-run errors (validation)	fail
V	Modern vision backbones $\times$ resolution (validation)	fail (best 0.65)
E	Larger same-family VLM (pilot)	do not scale up (27B: 2/20 correct)
E-video	8-frame clip vs single frame, local 8B	no help (0/20 in both arms)
K	Bundle-outlier baseline after [4] (descriptive)	most robust single detector (0.82–0.98)

Table 2: Gate timeline. Full pre-registrations and results are in the repository.

6 Results

6.1 The surrogate is a denoiser, not a better simulator

On fixed validation frames, the surrogate’s one-step RMSE is 4.85×10^{-5} , against 7.0×10^{-6} for constant velocity (Gate B). It is $6.9 \times$ worse than doing nothing on clean data. Its detection signal comes from denoising: a corrupted state that does not relax back leaves a residual in every frame. A clean-trained model ($\sigma = 0$) was indistinguishable from constant velocity in pilots.

6.2 Local failures: simple rules match or beat the surrogate

Table 3 gives test-set frame AUROC by event phase. Constant velocity is near-perfect at the onset of every failure, because any injected jump is a second-difference spike. It collapses on sustained frames (0.49–0.66), since the corrupted motion is smooth again. The surrogate recovers part of that gap (0.70–0.79 on hourglass, penetration and inversion), which is the restoring effect. But spatial rules are phase-invariant and recover more: the position Laplacian scores 0.79–0.94. In the pre-registered Gate C, the non-causal GNN shows **no benefit** over the best baseline (-0.058 [$-0.073, -0.042$]). The causal GNN passes ($+0.023$ [$+0.006, +0.040$]), entirely from frozen regions.

failure	phase	GNN	const-vel	pos. Lapl.	vel. z	inv. tet	CLIP kNN
hourglass	onset	1.00	1.00	0.85	0.97	0.72	0.57
	sustained	0.79	0.62	0.85	0.50	0.72	0.57
penetration	onset	0.99	1.00	0.79	0.93	0.56	0.52
	sustained	0.70	0.66	0.79	0.55	0.55	0.51
inversion	onset	1.00	1.00	0.94	0.99	0.84	0.52
	sustained	0.73	0.63	0.94	0.52	0.84	0.51
instability	onset	0.81	0.99	0.61	0.77	0.51	0.50
	sustained	0.90	1.00	0.74	0.92	0.60	0.52
frozen	onset	0.94	1.00	0.48	0.86	0.50	0.50
	sustained	0.99	0.49	0.63	0.85	0.51	0.50

Table 3: Test-set frame AUROC (92 simulations, mean over 3 severities) by event phase. Bold is the best in the row where it is unique. Regenerate with `scripts/phase_table.py`.

The frozen win does not survive a complete rule pool. After seeing the test results, we checked a velocity Laplacian (a node’s velocity minus its neighbours’ mean) on validation simulations 20–49. It scores **1.00** on sustained frozen frames at every severity, against the GNN’s 0.97–0.99 (Figure 3). This rule was not in the pre-registered pool, so Gate C stands as reported. But the post-hoc reading “learned physics catches kinematic errors” does not hold. Each local failure has some one-line rule that catches it: the second difference in time, the Laplacian of positions, or the Laplacian of velocities.

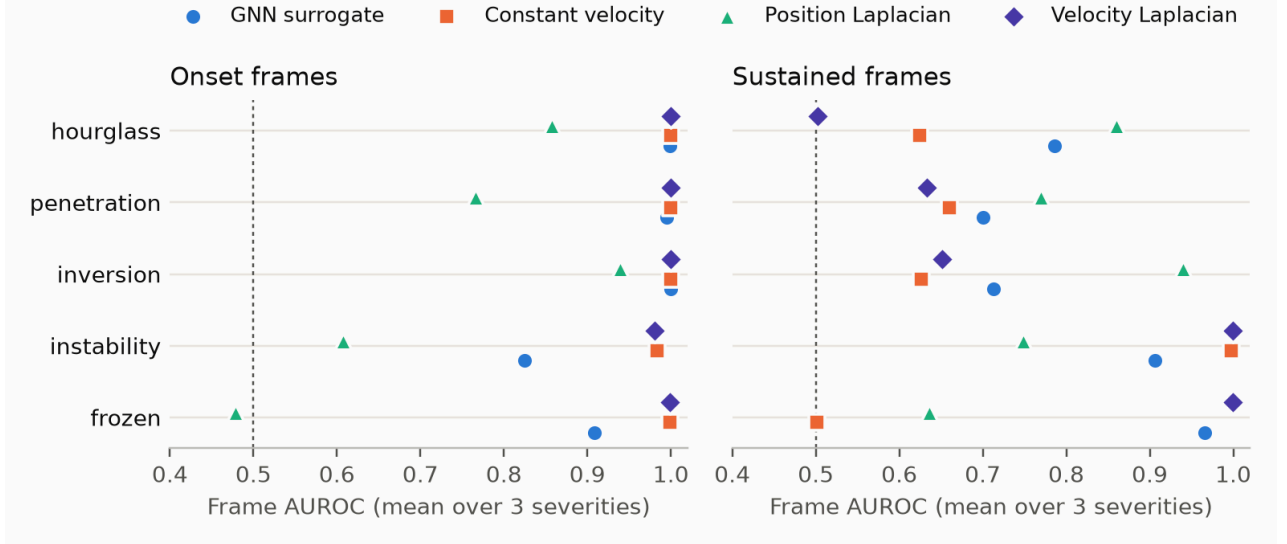


Figure 3: Onset vs sustained frame AUROC per detector (validation simulations 20–99, protocol v3, which includes the velocity Laplacian). The GNN never beats all three rules. On sustained frames, a different rule wins for each failure type.

A classical crash-data-mining baseline is stronger all-round. Following [4], we describe each frame by 18 mesh-independent statistics (quantiles of speed, acceleration, and position and velocity Laplacian residuals in units of the run’s step size, plus the inverted-element count and contact depth), and score it by its mean distance to the 5 nearest clean frames from validation simulations 50–69. On validation simulations 20–49 this bundle-outlier detector never collapses: frame AUROC is 0.82–0.98 in every type $\times$ phase cell. On sustained hourglass, penetration and inversion it scores 0.82, 0.88 and 0.91, against the GNN’s 0.79, 0.69 and 0.73. It doesn’t beat the best single rule in any cell, but no single rule is safe across all failure types, and the bundle is.

6.3 Whole-run loading errors: contact rules or nothing

Rules that look for local irregularities can’t see a run that is smooth everywhere but wrong for its loading. Gate D tested whether a surrogate can. The shape-from-load surrogate did not meet its pre-registered admission bar: final RMSE 1.99 cm against 3.18 cm for zero displacement, a ratio of 0.63 with a bar of 0.5. The velocity GNN never beat the best rule (all $\Delta < 0$). Contact rules detect response-scale errors (AUROC 0.85–1.00), time-scale errors (0.92–1.00) and 10-frame lag (0.91). **A 1–3 frame lag between actuator and plate is invisible to every detector** (AUROC 0.47–0.58). That is a measured detection limit for positions-only QA.

6.4 Vision: a modality limit, not a resolution limit

In Phase 1, CLIP patch-kNN stayed at 0.50–0.53 on penetration and inversion even at severity 3, where the shift is 16–20 px in a 448 px render. Gate V tested whether that is an artefact of an old backbone or low resolution. Table 4 shows the result. No condition reaches the pre-registered 0.80. DINOv2-L patch-kNN is the best (mean 0.65): it detects hourglassing (0.88 [0.79, 0.96]) but not penetration, inversion or frozen regions (0.55–0.62). An 896 px plate crop changes nothing. Two mechanisms plausibly contribute, and this study does not separate them. First, penetration and inversion happen on the actuator side of the plate, which the fixed camera does not see: the close-ups in Figure 1 had to be taken from that side. Second, the kNN memory bank holds clean renders of *other* simulations, whose geometry differs, so normal between-simulation variation competes with the anomaly. A same-simulation reference or multiple views might do better. As tested, renders lose much of the information the field-level rules use.

condition (sev. 3)	hourglass	penetration	inversion	instability	frozen	mean
CLIP B/16 kNN, 448	0.71	0.50	0.51	0.56	0.53	0.56
CLIP L/14 kNN, 448	0.67	0.55	0.54	0.54	0.52	0.56
DINOv2-L kNN, 448	0.88	0.55	0.58	0.65	0.59	0.65
DINOv2-L kNN, 896 crop	0.86	0.58	0.62	0.63	0.56	0.65
SigLIP2 zero-shot, 896 crop	0.66	0.52	0.53	0.56	0.52	0.56

Table 4: Gate V, validation simulations 20–39, frame AUROC at severity 3 (selected rows; full table in `results/gate_v`).

6.5 VLM explanations are not grounded

On 200 test frames sampled without filtering on detection, the local Qwen3-VL-8B called 199 frames “none” from images alone: 0% type accuracy on anomalies at every severity, including severity 3 (Table 5). Given numeric diagnostics, it reached 26%, labelling 72% of all frames “instability”. A depth-4 decision tree, cross-validated on the same diagnostics, reached 55%. The tree is supervised and the VLM zero-shot, so it marks what the diagnostics already contain rather than a like-for-like competitor.

arm (200 test frames)	type acc.	macro-F1	exact cell
Qwen3-VL-8B, images	12%	0.04	55%
Qwen3-VL-8B, images + diagnostics	26%	0.18	41%
decision tree on diagnostics (supervised, 5-fold)	55%	0.55	79%
majority class / majority cell	18%	0.05	49%

Table 5: VLM explanation correctness. Chance for 6 classes is 17%.

Counterfactual pairs. For 40 pairs, frame A is a clean, strongly bent frame and frame B is the same frame with a severity-3 hourglass (median visible shift 17 px; Figure 4).

- From the render alone, and with the heatmap, the VLM called every B frame “none”.
- With diagnostics, it separated 78% of pairs but never named “hourglass”.
- With diagnostics, the term overlap between its explanations of A and of B (Jaccard 0.25) equals that of unrelated pairs (0.24). From the render alone, it gave near-identical explanations for A and B (0.97).

The explanations are fluent but not grounded in what the model sees, which is an automation-bias risk in engineering sign-off.

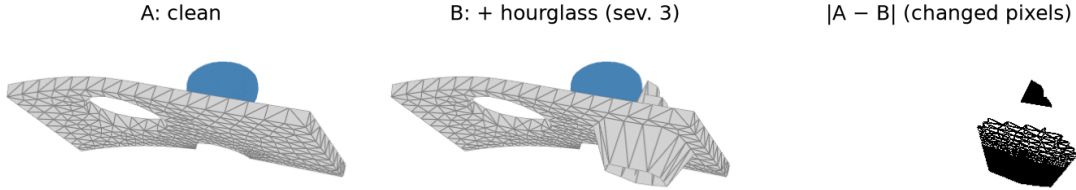


Figure 4: A counterfactual pair: clean frame A and the same frame with an injected severity-3 hourglass (B), with the changed pixels. The failure is plainly visible, yet from the render alone the 8B model answered “none” for both frames.

Does a larger model help? A pre-registered pilot ran Qwen3.8-27B (OpenRouter free tier) on 20 severity-3 anomalous frames, 5 clean frames and 10 counterfactual pairs, paired against the 8B on the same images and prompts.

- The 27B stops defaulting to “none” (6/20 anomalous frames, against 20/20 for the 8B).
- It localizes the residual hotspot: exact grid cell 11/20.
- But it names the right failure only 2/20 times, mostly answering “penetration”, plausibly because the heatmap is often red near the actuator.
- It separates 1 of 10 counterfactual pairs.

The pre-registered rule says do not scale up: the larger model sees *where* the surrogate flags a problem, but not *what* went wrong.

Does video help? Crash results are reviewed as animations, so a second pre-registered pilot gave the local 8B an 8-frame clip (frames $t-7 \dots t$, plain renders) instead of a single frame, on the same 25 frames and 10 pairs. It answered “none” to all 90 prompts in both arms: 0/20 correct and 0/10 pairs separated either way. Temporal context does not rescue a model that doesn’t see the failure in the first place.

6.6 Lead time

On the test set, with causal calibration and per-frame thresholds at 95% specificity set on validation, constant velocity warns 39 frames before a growing instability reaches its cap, the GNN 18 frames and the position Laplacian 7. These are not like-for-like: the realised pre-event false-alarm rates differ (6.2% for constant velocity, 1.1% for the GNN), and a stricter threshold would shorten constant velocity’s lead. A conformal run-level alarm with a guaranteed false-alarm rate was attempted. On validation, its false-alarm rate exceeded

the nominal α (up to 30% at $\alpha = 10\%$), so we do not claim the guarantee. We treat exchangeability across simulation ranges as an open question.

7 Discussion

Practitioner checklist.

1. Report a learned residual detector against its zero-output baseline (here, constant velocity) and cheap spatial rules. Here they matched or beat it for every failure type once a velocity rule was included.
2. Evaluate on sustained frames, not only onset: onset flatters every temporal detector.
3. When fields are available, check fields rather than renders. Vision backbones lose the signal even at higher resolution.
4. Don't use free-text VLM explanations for sign-off without grounding tests. Fluent answers were not tied to image content.

What is left for learned models is narrower than hoped: failures that are smooth, locally plausible and only inconsistent with the loading. Our shape surrogate did not reach the accuracy needed for that, and the smallest timing errors defeated every detector.

8 Limitations

The failures are injected, not produced by a solver. They snap back to the clean trajectory, and the dataset is one quasi-static hyperelastic benchmark, not explicit crash dynamics. The surrogate is laptop-scale: 8 h and 250 of 1,000 trajectories. The VLM study uses one model family (Qwen), and a cross-family pilot is pending. The replication set (training trajectories 250–349) has not yet been scored. Next steps are solver-generated failures, for example OpenRadioss runs with hourglass control disabled and labelled by solver energies; a real crash dataset once one is public; longer temporal context and multi-view renders; and simulation-grounded explanation methods.

Use of AI tools. An AI coding assistant (Anthropic's Claude) was used as a research-engineering tool to accelerate implementation, debugging, experiment monitoring and drafting, and to run analyses in parallel. The author designed the study, set every pre-registered decision rule, and reviewed and takes responsibility for all code, results and text.

References

- [1] N. McGreivy and A. Hakim, "Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations," *Nature Machine Intelligence*, vol. 6, pp. 1256–1269, 2024.
- [2] T. C. Spruegel, "Generic approach to plausibility checks for structural mechanics with deep learning," in *Proceedings of the Design Society*, [Online]. Available: <https://www.designsociety.org/publication/39529>
- [3] S. Bickel, S. Goetz, and S. Wartzack, "Detection of Plausibility and Error Reasons in Finite Element Simulations with Deep Learning Networks," *Algorithms*, vol. 16, no. 4, 2023.
- [4] D. Kracker, R. Dhanasekaran, A. Schumacher, and J. Garcke, "Method for automated detection of outliers in crash simulations," *International Journal of Crashworthiness*, vol. 28, no. 1, 2023.
- [5] M. S. Sarfraz, "Position: Quo Vadis, Unsupervised Time Series Anomaly Detection?," in *International Conference on Machine Learning (ICML)*, 2024.
- [6] R. Wu and E. Keogh, "Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress," *IEEE Transactions on Knowledge and Data Engineering*, 2023.
- [7] J. Jeong, "WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [8] K. Roth, "Towards Total Recall in Industrial Anomaly Detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] A. Radford, "Learning Transferable Visual Models From Natural Language Supervision," in *International Conference on Machine Learning (ICML)*, 2021.
- [10] M. Oquab, "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024.
- [11] X. Jiang, "MMAD: A Comprehensive Benchmark for Multimodal Large Language Models in Industrial Anomaly Detection," in *International Conference on Learning Representations (ICLR)*, 2025.

- [12] J. Ezemba, C. McComb, and C. Tucker, “Simulation vs. Hallucination: Assessing Vision-Language Model Question Answering Capabilities in Engineering Simulations,” in *Workshop on Design Automation for CPS and IoT (DESTION)*, 2025.
- [13] Mathieu, “Enhancing numerical simulation analysis with the use of explainable artificial intelligence and large language models: a case study on vehicle side crash optimization,” in *Proceedings of the Design Society*, 2025.
- [14] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, “Learning Mesh-Based Simulation with Graph Networks,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [15] M. Tschannen, “SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features,” *arXiv preprint arXiv:2502.14786*, 2025.
- [16] Qwen Team, “Qwen3-VL Technical Report,” 2025.